



使用json web token | 大前端

收藏人: KILLKISS

2015-05-27 | 阅: 151 转: 0 | 来源 | 分享

由来

做了这么长时间的web开发，从JAVA EE中的jsf, spring, hibernate框架，到spring web MVC，到用php框架thinkPHP，到现在的nodejs，我自己的看法是，这些框架提供了很大的便利，可以让开发人员专注于业务逻辑开发，这样其实就省去了前端开发的工作。而后来发现有些客户需要的展现形式很难实现，或者通

净整洁的web层，可以做到web层分离，通过ajax和服务端通信。现在在学习AngularJS框架，后台数据服务端打算用REST风格的接口来做，这个在前端

转载请注明出处: http://www.haomou.net/2014/08/13/2014_web_token/

来龙去脉

诸如Ember, Angular, Backbone之类的前端框架类库正随着更加精细的Web应用而日益壮大。正因如此，服务器端的组建也正正在从传统的任务中解

更大的便利。这种途径也使得一个移动应用和网页应用可以使用相同的后端。

当使用一个API时，其中一个挑战就是认证（authentication）。在传统的web应用中，服务端成功的返回一个响应（response）依赖于两件事。一是，一个字符串，它被用来让未来的请求（Request）检索信息。其次，包含在响应头（Header）里面的信息使客户端保存了一个Cookie。服务器自动的在每个子向无连接的方法（This is how traditional web applications get around the fact that HTTP is stateless）。

API应该被设计成无状态的（Stateless）。这意味着没有登陆，注销的方法，也没有sessions，API的设计者同样也不能依赖Cookie，因为不能保证这些个可能的解决这个问题的机制。这篇文章利用Node的Express框架作为后端，以及Backbone作为前端。

常用方法

第一个是使用在HTTP规范中所制定的Basic Auth，它需要在在响应中设定一个验证身份的Header。客户端必须在每个子响应是附加它们的凭证（credentials）。

第二个方面有点类似，但是使用应用自己的验证机制。通常包括将发送的凭证与存储的凭证进行检查。和Basic Auth相比，这种需要在每次请求（call）时都

第三种是OAuth（或者OAuth2）。为第三方的认证所设计，但是更难配置。至少在服务器端更难。

在使用中，并不会每次都让用户提交用户名和密码，通常的情况是客户端通过一些可靠信息和服务器交换取token，这个token作为客户端再次请求的权限API的时候都要附加上它。然后，这仍然比直接发送账户和密码更加安全，哪怕是HTTPS。

把token想象成一个安全的护照。你在一个安全的前台验证你的身份（通过你的用户名和密码），如果你成功验证了自己，你就可以取得这个。当你走到

JWTs

JWTs是一份草案，尽管在本质上它是一个老生常谈的一种更加具体的认证授权的机制。一个JWT被周期（period）分成了三个部分。JWT是URL-safe的，参考 <http://self-issued.info/docs/draft-ietf-oauth-json-web-token.html>

JWT的第一部分是对一个简单js对象的编码后的字符串，这个js对象是用来描述这个token类型以及使用的hash算法。下面的例子展示的是一个使用了HMAC

```
1 {
2   "typ" : "JWT",
3   "alg" : "HS256"
4 }
```

在加密之后，这个对象变成了一个字符串：

eyJ0eXAiOiJKV1QiLA0KICJhbGciOiJIUzI1NiJ9

JWT的第二部分是token的核心，这部分同样是对一个js对象的编码，包含了一些摘要信息。有一些是必须的，有一些是选择性的。实例如下：

```
1 {
2   "iss": "joe",
3   "exp": 1300819380,
4   "http://example.com/is_root": true
5 }
```

```
5 }
```

这个结构被称为**JWT Claims Set**。这个**iss**是**issuer**的简写，表明请求的实体，可以是发出请求的用户的信息。**exp**是**expires**的简写，是用来指定**token**的

```
1 eyJpc3MiOiJqb2UiLA0KICJleHAiOiEzMDA4MTkzODAsDQogImh0dHA6Ly9leGFtcGxlLmNvbS9pc19yb290Ijpb0cnVlQ
```

JWT的第三个部分，是**JWT**根据第一部分和第二部分的签名（**Signature**）。像这个样子：

```
1 dBjftJez4CVP-mB92K27uhbUJU1plr_wW1gFWFOEjXk
```

最后将上面的合并起来，**JWT token**如下：

```
1 eyJ0eXAiOiJKV1QiLA0KICJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJqb2UiLA0KICJleHAiOiEzMDA4MTkzODAsDQogImh0dHA6Ly9leGFtcGxlLmNvbS9pc19yb290Ijpb0cnVlQ
```

处理**Tokens**

我们将用**JWT simple**模块去处理**token**，它将使我们从钻研如何加密解密中解脱出来。如果你有兴趣，可以阅读这篇说明，或者读这个仓库的源码。首先我们将使用下面的命令安装这个库。记住你可以在命令中加入**—save**，让其自动的让其加入到你的**package.json**文件里面。

```
npm install jwt-simple
```

在你应用的初始环节，加入以下代码。这个代码引入了**Express**和**JWT simple**，而且创建了一个新的**Express**应用。最后一行设定了**app**的一个名为**jwtTokenSecret**

```
var express = require('express');
var jwt = require('jwt-simple');
var app = express();
```

```
app.set('jwtTokenSecret', 'YOUR_SECRET_STRING');
```

获取**token**

我们需要做的第一件事就是让客户端通过他们的账号密码交换**token**。这里有2种可能的方法在**RESTful API**里面。第一种是使用**POST**请求来通过验证，好的使用请求头。

这篇文章的目的是为了解释**token**验证的方法而不是基本的用户名/密码验证机制。所以我们假设我们已经通过请求得到了用户名和密码：

```
User.findOne({ username: username }, function(err, user) {
  if (err) {
    // user not found
    return res.send(401);
  }

  if (!user) {
    // incorrect username
    return res.send(401);
  }

  if (!user.validPassword(password)) {
    // incorrect password
    return res.send(401);
  }

  // User has authenticated OK
  res.send(200);
});
```

如果用户成功验证账号和密码，然后我们生成一个**token**，返回给用户。

```
var expires = moment().add('days', 7).valueOf();
var token = jwt.encode({
  iss: user.id,
  exp: expires
}, app.get('jwtTokenSecret'));

res.json({
  token: token
});
```

```

    token: token,
    expires: expires,
    user: user.toJSON()
  });

```

注意到`jwt.encode()`函数有2个参数。第一个就是一个需要加密的对象，第二个是一个加密的密钥。这个`token`是由我们之前提到的`iss`和`exp`组成的。注意

验证Token

客户端获取到`token`后，应该在每次向服务器请求数据时附带这个`token`，然后服务端验证`token`。

为了验证JWT，我们需要写出一些可以完成这些功能的中间件（Middleware）：

检查附上的`token`

试图解密

验证`token`的可用性

如果`token`是合法的，检索里面用户的信息，以及附加到请求的对象上

我们来写一个中间件的框架

```

// @file jwtauth.js

var UserModel = require('../models/user');
var jwt = require('jwt-simple');
1
module.exports = function(req, res, next) {
  // code goes here
};

```

为了获得最大的可扩展性，我们允许客户端使用一下3个方法附加我们的`token`：作为请求链接（`query`）的参数，作为主体的参数（`body`），和作为请求

下面是我们的允许在中间件的代码，试图去检索`token`：

```

1 var token = (req.body && req.body.access_token) || (req.query && req.query.access_token) || req.head

```

注意到他为了访问`req.body`，我们需要首先使用`express.bodyParser()`中间件（译者注，这个是Express 3.x的中间件）。

下一步，我们讲解析JWT：

```

if (token) {
  try {
    var decoded = jwt.decode(token, app.get('jwtTokenSecret'));

    // handle token here
1 } catch (err) {
    return next();
  }
} else {
  next();
}

```

如果解析的过程失败，那么JWT Simple组件将会抛出一段异常。如果异常发生了，或者没有`token`，我们将会调用`next()`来继续处理请求。这代表喆我们的时间戳。我们将首先处理后者，如果它过期了，我们就拒绝它：

```

if (decoded.exp <= Date.now()) {
1 res.end('Access token has expired', 400);
}

```

如果`token`依旧合法，我们可以从中检索出用户信息，并且附加到请求对象里面去：

```

User.findOne({ _id: decoded.iss }, function(err, user) {
1 req.user = user;
});

```

最后，将这个中间件附加到路由里面：

```
var jwtauth = require('./jwtauth.js');

1 app.get('/something', [express.bodyParser(), jwtauth], function(req, res){
  // do something
});
```

或者匹配一些路由

```
1 app.all('/api/*', [express.bodyParser(), jwtauth]);
```

客户端请求

我们提供了一个简单的get端去获得一个远端的token。这非常直接了，所以我们不用纠结细节，就是发起一个请求，传递用户名和密码，如果请求成功

我们现在研究的是后续的请求。一个方法是通过jQuery的ajaxSetup()方法。这可以直接用来做Ajax请求，或者通过前端框架使用包装过的Ajax方法。比Storage）里面，我们可以通过这种方法将token附加到请求头里面：

```
var token = window.localStorage.getItem('token');

if (token) {
  $.ajaxSetup({
1    headers: {
      'x-access-token': token
    }
  });
}
```

很简单，但是这会劫持所有Ajax请求，如果这里有一个token在本地存储里面。它将会附加到一个名为x-access-token的Header里面。

bear token

关于bear token，参看 [RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage](#)，目前国内各大网站都是用不同的token，也没说必须使

OAuth 2.0 (RFC 6749) 定义了 Client 如何取得 Access Token 的方法。Client 可以用 Access Token 以 Resource Owner 的名义来向 Resource Server 取 Facebook (Resource Server) 取得我的朋友名单 (Protected Resource)。OAuth 2.0 定义Access Token 是 Resource Server 用来认证的唯一方式，有了这

然而在 RFC 6749 里面只定义抽象的概念，细节如 Access Token 格式、怎么传到 Resource Server，以及 Access Token 无效时，Resource Server 怎 Authorization Server 在 Resource Owner 的允许下核发给 Client，Resource Server 只要认在这个 Token 就可以认定 Client 已经获取 Resource Owner 的

Bearer Token 的格式

```
1 Bearer XXXXXXXXX
```

其中 XXXXXXXXX 的格式为 b64token，ABNF 的定义：

```
1 b64token = 1*( ALPHA / DIGIT / "-" / "." / "_" / "~" / "+" / "/" ) *
```

写成 Regular Expression 即是：

```
1 /[A-Za-z0-9\-\.\_\~\+\ \/ ]+="/
```

关于Bear Token还是打算另起一篇，详细说明：[Bearer Token](#)

express-jwt实例

下面给一个具体的实例，这个例子的客户端是web app，使用AngularJS框架。服务端使用NodeJS做的RESTful API接口，客户端直接调用接口数据，当用户把他的授权信息发过来的时候，Node.js 服务检查是否正确，然后返回一个基于用户信息的唯一 token。AngularJS 应用把 token 保存在用户的认用户授权，服务端检查验证这个 token，然后如果成功了就返回数据，如果失败了返回 401 或者其它的异常。

用到的技术：

AngularJS

NodeJS （express.js, express-jwt 和 mongoose）

MongoDB

“ ” 多 用 于 开 发 者 可 以 通 过 这 个 库 来 实 现 这 个 功 能 。

Redis (备用, 用于记录用户退出登录时候还没有超时的token)

客户端 : AngularJS 部分

首先, 我们来创建我们的 AdminUserCtrl controller 和处理 login/logout 动作。

```
1 appControllers.controller('AdminUserCtrl', ['$scope', '$location', '$window', 'UserService', 'Authen
2     function AdminUserCtrl($scope, $location, $window, UserService, AuthenticationService) {
3
4         //Admin User Controller (login, logout)
5         $scope.logIn = function logIn(username, password) {
6             if (username !== undefined && password !== undefined) {
7
8                 UserService.logIn(username, password).success(function(data) {
9                     AuthenticationService.isLogged = true;
10                     $window.sessionStorage.token = data.token;
11                     $location.path("/admin");
12                 }).error(function(status, data) {
13                     console.log(status);
14                     console.log(data);
15                 });
16             }
17         }
18
19         $scope.logout = function logout() {
20             if (AuthenticationService.isLogged) {
21                 AuthenticationService.isLogged = false;
22                 delete $window.sessionStorage.token;
23                 $location.path("/");
24             }
25         }
26     }
27 ]);
```

这个 controller 用了两个 service: UserService 和 AuthenticationService。第一个处理调用 REST api 用证书。后面一个处理用户的认证。它只有一个布

```
1 appServices.factory('AuthenticationService', function() {
2     var auth = {
3         isLogged: false
4     }
5
6     return auth;
7 });
8 appServices.factory('UserService', function($http) {
9     return {
10         logIn: function(username, password) {
11             return $http.post(options.api.base_url + '/login', {username: username, password: passw
12         },
13
14         logOut: function() {
15
16         }
17     }
18 });
```

好了, 我们需要做张登陆页面:

```
<form class="form-horizontal" role="form">
  <div class="form-group">
    <label for="inputUsername" class="col-sm-4 control-label">Username</label>
    <div class="col-sm-4">
      <input type="text" class="form-control" id="inputUsername" placeholder="Username" ng-moc
    </div>
  </div>
  <div class="form-group">
```

```

    <label for="inputPassword" class="col-sm-4 control-label">Password</label>
1    <div class="col-sm-4">
        <input type="password" class="form-control" id="inputPassword" placeholder="Password" ng
    </div>
</div>
<div class="form-group">
    <div class="col-sm-offset-4 col-sm-10">
        <button type="submit" class="btn btn-default" ng-click="login(login.email, login.passwor
    </div>
</div>
</form>

```

当用户发送他的信息过来，我们的 **controller** 把内容发送到 **Node.js** 服务器，如果信息可用，我们把 **AuthenticationService** 里面的 **isLogged** 设为 **true**。

好了，我们要往每个请求里面追加一个特殊的头信息了:[**Authorization: Bearer**]。为了实现这个需求，我们建立一个服务，叫 **TokenInterceptor**。

```

1 appServices.factory('TokenInterceptor', function ($q, $window, AuthenticationService) {
2     return {
3         request: function (config) {
4             config.headers = config.headers || {};
5             if ($window.sessionStorage.token) {
6                 config.headers.Authorization = 'Bearer ' + $window.sessionStorage.token;
7             }
8             return config;
9         },
10
11        response: function (response) {
12            return response || $q.when(response);
13        }
14    };
15 });

```

然后我们把这个 **interceptor** 追加到 **\$httpProvider**：

```

1 app.config(function ($httpProvider) {
2     $httpProvider.interceptors.push('TokenInterceptor');
3 });

```

然后，我们要开始配置路由了，让 **AngularJS** 知道哪些需要授权，在这里，我们需要检查用户是否已经被授权，也就是查看 **AuthenticationService** 的 **is**

```

1 app.config(['$locationProvider', '$routeProvider',
2     function($location, $routeProvider) {
3         $routeProvider.
4             when('/', {
5                 templateUrl: 'partials/post.list.html',
6                 controller: 'PostListCtrl'
7             }).
8             when('/post/:id', {
9                 templateUrl: 'partials/post.view.html',
10                controller: 'PostViewCtrl'
11            }).
12            when('/tag/:tagName', {
13                templateUrl: 'partials/post.list.html',
14                controller: 'PostListTagCtrl'
15            }).
16            when('/admin', {
17                templateUrl: 'partials/admin.post.list.html',
18                controller: 'AdminPostListCtrl',
19                access: { requiredLogin: true }
20            }).
21            when('/admin/post/create', {
22                templateUrl: 'partials/admin.post.create.html',
23                controller: 'AdminPostCreateCtrl',
24                access: { requiredLogin: true }
25            }).

```

```
26     when('/admin/post/edit/:id', {
27         templateUrl: 'partials/admin.post.edit.html',
28         controller: 'AdminPostEditCtrl',
29         access: { requiredLogin: true }
30     }).
31     when('/admin/login', {
32         templateUrl: 'partials/admin.login.html',
33         controller: 'AdminUserCtrl'
34     }).
35     when('/admin/logout', {
36         templateUrl: 'partials/admin.logout.html',
37         controller: 'AdminUserCtrl',
38         access: { requiredLogin: true }
39     }).
40     otherwise({
41         redirectTo: '/'
42     });
43 }]);
44
45 app.run(function($rootScope, $location, $window, AuthenticationService) {
46     $rootScope.$on("$routeChangeStart", function(event, nextRoute, currentRoute) {
47         //redirect only if both isLoggedIn is false and no token is set
48         if (nextRoute !== null && nextRoute.access !== null && nextRoute.access.requiredLogin
49             && !AuthenticationService.isLoggedIn && !$window.sessionStorage.token) {
50
51             $location.path("/admin/login");
52         }
53     });
54 });
```

服务端: Node.js + MongoDB 部分

为了在我们的 RESTful api 处理授权信息, 我们要用到 **express-jwt** (JSON Web Token) 来生成一个唯一 Token, 基于用户的信息。以及验证 Token。

首先, 我们在 **MongoDB** 里面创建一个用户的 **Schema**。我们还要创建调用一个中间件, 在创建和保存用户信息到数据库之前, 用于加密密码。还有我

```
1 var Schema = mongoose.Schema;
2
3 // User schema
4 var User = new Schema({
5     username: { type: String, required: true, unique: true },
6     password: { type: String, required: true }
7 });
8
9 // Bcrypt middleware on UserSchema
10 User.pre('save', function(next) {
11     var user = this;
12
13     if (!user.isModified('password')) return next();
14
15     bcrypt.genSalt(SALT_WORK_FACTOR, function(err, salt) {
16         if (err) return next(err);
17
18         bcrypt.hash(user.password, salt, function(err, hash) {
19             if (err) return next(err);
20             user.password = hash;
21             next();
22         });
23     });
24 });
25
26 // Password verification
27 User.methods.comparePassword = function(password, cb) {
28     bcrypt.compare(password, this.password, function(err, isMatch) {
29         if (err) return cb(err);
```

```
30     cb(isMatch);
31   });
32 };
```

然后我们开始写授权用户和创建 **Token** 的方法:

```
1 exports.login = function(req, res) {
2   var username = req.body.username || '';
3   var password = req.body.password || '';
4
5   if (username == '' || password == '') {
6     return res.send(401);
7   }
8
9   db.userModel.findOne({username: username}, function (err, user) {
10     if (err) {
11       console.log(err);
12       return res.send(401);
13     }
14
15     user.comparePassword(password, function(isMatch) {
16       if (!isMatch) {
17         console.log("Attempt failed to login with " + user.username);
18         return res.send(401);
19       }
20
21       var token = jwt.sign(user, secret.secretToken, { expiresInMinutes: 60 });
22
23       return res.json({token:token});
24     });
25
26   });
27 };
```

最后, 我们需要把 **jwt** 中间件加到所有的, 访问时需要授权的路由上面:

```
1 /*
2  Get all published posts
3  */
4 app.get('/post', routes.posts.list);
5 /*
6   Get all posts
7  */
8 app.get('/post/all', jwt({secret: secret.secretToken}), routes.posts.listAll);
9
10 /*
11   Get an existing post. Require url
12  */
13 app.get('/post/:id', routes.posts.read);
14
15 /*
16   Get posts by tag
17  */
18 app.get('/tag/:tagName', routes.posts.listByTag);
19
20 /*
21   Login
22  */
23 app.post('/login', routes.users.login);
24
25 /*
26   Logout
27  */
28 app.get('/logout', routes.users.logout);
29
```

```
29
30 /*
31     Create a new post. Require data
32 */
33 app.post('/post', jwt({secret: secret.secretToken}), routes.posts.create);
34
35 /*
36     Update an existing post. Require id
37 */
38 app.put('/post', jwt({secret: secret.secretToken}), routes.posts.update);
39
40 /*
41     Delete an existing post. Require id
42 */
43 app.delete('/post/:id', jwt({secret: secret.secretToken}), routes.posts.delete);
```

上面这个实例就采用了token的验证方式构建了api接口，但是有两个问题需要解决：

用户退出登录，但是token并没有失效，因为服务端没有删除这个token

token失效了，怎么办，如果还是让用于登录重新获取token，会体验不好。应该有token刷新机制。

使用Redis解决问题1

解决方法是：当用户点了logout按钮的时候，Token只会保存一段时间，就是你用jsonwebtoken登陆之后，token有效的这段时间，我们将这个token作为一个nodejs的中间件，检查所有受限endpoint用的token是否存在Redis数据库中。

NodeJS 配置 Reids

```
1 var redis = require('redis');
2 var redisClient = redis.createClient(6379);
3
4 redisClient.on('error', function (err) {
5     console.log('Error ' + err);
6 });
7
8 redisClient.on('connect', function () {
9     console.log('Redis is ready');
10 });
11
12 exports.redis = redis;
13 exports.redisClient = redisClient;
```

然后，我们来创建一个方法，用来检查提供的token是不是被

Token 管理和中间件

为了在Redis中保存Token，我们要创建一个方法来拿到请求中的Header的Token参数，然后把它作为Redis的key保存起来。值是什么我们不管它

```
1 var redisClient = require('./redis_database').redisClient;
2 var TOKEN_EXPIRATION = 60;
3 var TOKEN_EXPIRATION_SEC = TOKEN_EXPIRATION * 60;
4
5 exports.expireToken = function(headers) {
6     var token = getToken(headers);
7
8     if (token != null) {
9         redisClient.set(token, { is_expired: true });
10        redisClient.expire(token, TOKEN_EXPIRATION_SEC);
11    }
12 };
13
14 var getToken = function(headers) {
15     if (headers && headers.authorization) {
16         var authorization = headers.authorization;
17         var part = authorization.split(' ');
```

```

17         var part = authorization.split(' ');
18
19         if (part.length == 2) {
20             var token = part[1];
21
22             return part[1];
23         }
24         else {
25             return null;
26         }
27     }
28     else {
29         return null;
30     }
31 };

```

然后，再创建一个中间件来验证一下 **token**，当用户发起请求的时候：

```

1 // Middleware for token verification
2 exports.verifyToken = function (req, res, next) {
3     var token = getToken(req.headers);
4
5     redisClient.get(token, function (err, reply) {
6         if (err) {
7
8             console.log(err);
9             return res.send(500);
10
11         }
12         if (reply) {
13             res.send(401);
14         }
15         else {
16             next();
17         }
18     });
19 };

```

verifyToken 这个方法，是一个中间件，用来拿到请求头中的 **token**，然后在 **Redis** 里面查找它。如果 **token** 被发现了，我们就发 **HTTP 401**。否则我们就：

我们要在用户点 **logout** 的时候，执行 **expireToken** 方法：

```

1 exports.logout = function(req, res) {
2     if (req.user) {
3         tokenManager.expireToken(req.headers);
4
5         delete req.user;
6         return res.send(200);
7     }
8     else {
9         return res.send(401);
10    }
11 }

```

最后我们更新路由，用上新的中间件：

```

1 //Login
2 app.post('/user/signin', routes.users.signin);
3
4 //Logout
5 app.get('/user/logout', jwt({secret: secret.secretToken}), routes.users.logout);
6
7 //Get all posts
8 app.get('/post/all', jwt({secret: secret.secretToken}), tokenManager.verifyToken, routes.posts.list

```

```
9
10 //Create a new post
11 app.post('/post', jwt({secret: secret.secretToken}), tokenManager.verifyToken , routes.posts.create
12
13 //Edit the post id
14 app.put('/post', jwt({secret: secret.secretToken}), tokenManager.verifyToken, routes.posts.update);
15
16 //Delete the post id
17 app.delete('/post/:id', jwt({secret: secret.secretToken}), tokenManager.verifyToken, routes.posts.c
```

好了，现在我们每次发送请求的时候，我们都去解析 **token**，然后看看是不是有效的。

这里有整个项目的[源代码](#)

refresh token解决问题2

```
1 appServices.factory('TokenInterceptor', function ($q, $window, $location, AuthenticationService) {
2     return {
3         request: function (config) {
4             config.headers = config.headers || {};
5             if ($window.sessionStorage.token) {
6                 config.headers.Authorization = 'Bearer ' + $window.sessionStorage.token;
7             }
8             return config;
9         },
10
11         requestError: function(rejection) {
12             return $q.reject(rejection);
13         },
14
15         /* Set Authentication.isAuthenticated to true if 200 received */
16         response: function (response) {
17             if (response != null && response.status == 200 && $window.sessionStorage.token && !Auth
18                 AuthenticationService.isAuthenticated = true;
19             }
20             return response || $q.when(response);
21         },
22
23         /* Revoke client authentication if 401 is received */
24         responseError: function(rejection) {
25             if (rejection != null && rejection.status === 401 && ($window.sessionStorage.token || ?
26
27                 delete $window.sessionStorage.token;
28                 AuthenticationService.isAuthenticated = false;
29                 $location.path("/admin/login");
30             }
31             return $q.reject(rejection);
32         }
33     };
34 });
```

上面代码中的最后一部分**responseError**其实就是授权失败的部分，这里面的处理方法是返回到登录授权页面。

这里面考虑的方法是，如果是**token**超时，使用**refresh_token**来换取新的**token**。这个**refresh_token**，是一开始核发的时候一块发布给客户端的，这里就

思路1：在**user**中记录**token**超时时间，计算一下剩余时间，如果剩余时间比如说小于1分钟，开始核发新的**token**，客户端自动使用新的**token**，等退出时

来自: [KILL KISS](#) > 《架构》

上一篇: [通向架构师的道路（第五天）之tomcat集群](#)

现货原油骗局	徕卡相机大全	上海宠物狗市场	狗狗托运价格		
巨型阿拉斯加犬	买房子首付多少	空运狗	oca贴合机	宠物托运价格	
出纳工作流程	宠物托运	徕卡相机	空运宠物多少钱	长途搬家公司	狗狗托运

类似文章

- 外星人就在月球背面
- 推荐:美国小学生怎样写作文
- 欧洲十大最美丽的古堡 艳羨世人
- 关乎微笑, 关乎幸福
- 唐太宗"一字千金"求的王羲之书法[16P]...
- 手相算命: 婚姻与爱情
- 感悟老子《道德经》
- 诗情画意—话冬至

热门推广

- 1 TOP:白富美是这样做月子的
- 2 网站建设,企业整站个性化定制
- 3 爱贝少儿英语,值得信赖的品牌!
- 4 写字难看大问题!极大影响人生
- 5 立见奇效,写出一手漂亮好字!
- 6 21天练出一手好字,练字必备!
- 7 小夫妻在家做手工 月赚50000元
- 8 易管家具软件推出新版本E8

猜你喜欢

换一组

 50个让人落泪的小段子	 教你如何拍出漂亮女生	 9种食物吸光你的脂肪,"吃"竟然也能..	 借酒力,对女人说几句心里话	 最富哲理的"说文解字"
 太全啦煲粥·不用再学了	 人像摄影入门技巧	 青春永驻—"天泽丹"	 儿媳写给婆婆的信	 千万记住了啊!房子千万不要这样装...

发表评论:

您好, 请 [登录](#) 或者 [注册](#) 后再进行评论

其它帐号登录: